

SQL for Quant Interviews: The Complete Guide

From Data Modeling to Query Plans

Production-Grade Patterns for Research, Risk, and Trading

Amit Kumar Jha

January 26, 2026

Module 2: Types, Precision, and Time

Module Overview

What this module is about:

In quant SQL, correctness errors come from types more than from syntax: floating-point money, implicit casts, and ambiguous time. You should be able to articulate design rules that prevent entire classes of bugs.

Money: Never Use Binary Floating-Point for Accounting

Core Concept: Fixed-Point Representation

If an amount has d decimal places, store it as an integer:

$$\text{amount_int} = \lfloor 10^d \cdot \text{amount} \rfloor \quad \text{amount} = \frac{\text{amount_int}}{10^d}$$

Practitioner Insight

Many trading systems store quantities and prices as scaled integers (“price in ticks”, “qty in lots”, “notional in cents”) because this guarantees deterministic aggregation and avoids float rounding drift across systems. Warehouses often use `NUMERIC(p, s)` for convenience, but the conceptual model is still fixed-point.

Warning Sign

Using `FLOAT` for money can produce non-reproducible reconciliation breaks: sums depend on execution order, parallelism, and engine implementation. If a desk cares about cents, treat money as fixed-point.

Timestamps: Three Clocks You Must Separate

Clock	Meaning
<code>event_time</code>	when the market event occurred (trade/quote timestamp)
<code>ingestion_time</code>	when your system observed/received it (pipeline latency)
<code>asof_time</code>	when you evaluate reference data (EOD close, risk cutoff)

Why it matters

In backtests, you must join signals (`event_time`) to prices that were knowable at that time. Using `ingestion_time` can accidentally include late-corrected data. Using `asof_time` incorrectly can impose a valuation cutoff that belongs to reporting, not trading.

Time Zones and Trading Days

Warning Sign

A trading day is not the same as a UTC date. For many markets, “day” is a business-calendar concept with session boundaries, holidays, and daylight saving changes. Storing only `DATE(ts)` is a common source of wrong daily P&L.

Robust pattern:

- store **timestamps in UTC** at ingestion,

Module Quick Sheet

Mnemonic

SIP: Scan, Index/Prune, Plan.

Module Quick Sheet

Performance checklist (desk-realistic):

- Is the engine scanning too much? (partitions, indexes, clustering, pruning)
- Is it shuffling/sorting too much? (join keys, skew, window sorts)
- Are estimates wrong? (stale stats, bad histograms, correlated predicates)
- Are you materializing expensive intermediates? (CTE behavior, temp spills)
- Can you change the **shape** of work? (pre-aggregate, reduce columns early, filter earlier)

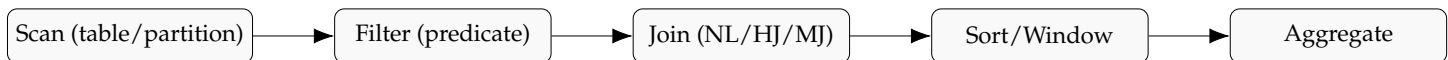
High-leverage optimization tricks:

- **Sargability:** avoid functions on indexed/partitioned columns in WHERE.
- **Covering indexes:** include projected columns to avoid table fetches (OLTP engines).
- **Composite index order:** match leading columns to the most selective predicates.
- **Partition pruning:** design predicates that constrain the partition key.
- **Cardinality reduction:** pre-aggregate facts before joining to dimensions.

Warning Sign

Interview warning signals:

- “Add an index” without specifying *which predicate* and *which access path* it enables.
- Not distinguishing OLTP tuning (indexes) from warehouse tuning (partitioning/clustering/shuffle).
- Ignoring data skew (one instrument/book dominates) which breaks parallel performance.



Typical “hot spots”:
scans, joins, sorts

Figure 7: A mental model of query work: make the expensive operators smaller or cheaper.

Realistic interview prompts

- “This query is slow; what do you check first?” (EXPLAIN, scan volume, join type, sort spills.)
- “Why did an index not help?” (not selective, wrong leading columns, function-wrapped predicate.)
- “How do you speed up an as-of join?” (index/cluster on (id, ts), reduce candidate rows, lateral join.)

Drills (with short solutions)

Drills + Short Solutions

How to use: Write your first draft without running it. Then validate (i) row-count invariants, (ii) join grain, (iii) time cutoff (no look-ahead).

Assumption: Postgres-style SQL; adapt minor syntax (e.g., DATE_TRUNC, QUALIFY) to your engine.

1. **Prompt:** Make a time filter sargable. Rewrite WHERE DATE(ts)=DATE '2026-01-01' to use an index on ts.

Common trap: Wrapping indexed columns in functions prevents index usage.

Solution (sketch):

```
1 | WHERE ts >= TIMESTAMP '2026-01-01'
```

Module 11: SQL for Backtesting (Leakage, Bias, and Hygiene)

Module Overview

What this module is about:

A quant backtest is a sequence of point-in-time queries. Most backtests fail quietly due to subtle leakage. This module lists the SQL-level failure modes and provides defensible patterns.

Leakage Patterns Unique to SQL

Common mistakes:

- joining to “latest” dimension snapshot rather than effective-dated snapshot,
- using close prices when decisions are made before close,
- inadvertently using corrected data that was not available historically,
- inner joins that drop missing data selectively (creating survivorship).

Warning Sign

An inner join can create survivorship bias: instruments with missing fundamentals disappear from the sample, and your backtest silently overweights “well-covered” names. Prefer left joins plus explicit missingness handling.

Decision Time vs Evaluation Time

Core Concept: Two-time framework

t_d = decision time, t_e = evaluation time

Signals must use only information with timestamp $\leq t_d$. P&L uses prices at t_e .

Executable Script: signal_and_pnl.sql

```
1  -- Sketch: generate signal at open, evaluate at close
2  WITH signal AS (
3      SELECT
4          symbol,
5          trading_day,
6          -- feature computed from history up to market open
7          mom AS signal_value
8      FROM features_open
9  ),
10 pnl AS (
11     SELECT
12         symbol,
13         trading_day,
14         close_px / open_px - 1 AS ret_oc
15     FROM ohlc_daily
16 )
17 SELECT s.symbol, s.trading_day, s.signal_value, p.ret_oc
18 FROM signal s
19 JOIN pnl p
20     ON p.symbol = s.symbol
21     AND p.trading_day = s.trading_day;
```